Locksmith Lab

Advice

Nikola Panayotov

Table of Contents

1 Introduction 2
2 Basis for the recommendation 2
3 Technology comparison 2
4 Proposed solution 3
4.1 Application structure 3
4.2 Environments and deployment 3
4.3 Security testing and hardening approach 4
4.4 Priority order 4
5 Quality and Optional Requirements 4
6 Design and implementation evidence 5
7 Validation and completion 6
8 Use of AI during the project 6
9 Sources 6

1 Introduction

I recommend building SecureNotes with React and TypeScript on the frontend, Node.js with Express and TypeScript on the backend, and PostgreSQL as the database, all run with Docker on Ubuntu Linux. Version 1 (Unsecured), the intentionally weak version, runs in the Fontys ICT Netlab. Version 2 (Secured), the hardened version, runs on my Ubuntu VPS behind Nginx with HTTPS and a UFW firewall. This fits my choice of a lighter backend stack and leaves the main learning effort for cybersecurity and networking. The project should deliver a complete build, attack, fix and retest cycle before any optional features are added. This advice uses the scope and requirements defined in Locksmith Lab Analysis and provides the proposed solution and global LO2 planning. It answers the question: what technical approach should be used for SecureNotes, and why is it suitable for the Security Playground challenge?

2 Basis for the recommendation

SecureNotes is a small web application with accounts, login, personal notes, a database and a network-facing API. The implementation must support realistic weaknesses in Version 1 and realistic fixes in Version 2, and it must run in two environments with different security settings. A successful choice should support these needs without requiring unnecessary learning outside the project goals.

I use a qualitative multi-criteria comparison. Time left for security and networking work carries the most weight, followed by my existing knowledge and the support the framework gives for the planned tests and fixes: authentication, authorization, password hashing, SQL injection, cross-site scripting, sessions, rate limiting, logging and HTTPS. Control over how each weakness is introduced and repaired also matters, because seeing the cause of a vulnerability is a learning goal. The comparison is a project judgment, not a measured benchmark.

3 Technology comparison

Express fits my goals most closely because it is simple, widely used and flexible for building REST APIs, so most project time can go to testing and hardening. Fastify is a valid alternative. It includes JSON Schema validation as part of the framework [3], so the issue is not a lack of capability but that Express is the one I chose for its simplicity and wide documentation [1]. Java with Spring Boot was considered at the start of the project. It would support every planned test, but learning a heavier backend framework would reduce the time for the security work [4].

Criterion	Node.js with Express	Node.js with Fastify	Java with Spring Boot
Learning effort	Small, and well documented. Leaves time for security work.	Small, with a smaller volume of general tutorials.	Larger, because of the heavier framework and project structure.
Input validation	Not built in. Added through libraries such as Zod, Joi or express-validator, and must be tested deliberately.	Built-in schema validation [3].	Available through the framework ecosystem.
Security middleware	Added with Helmet, express-rate-limit, cookie-parser and cors [2].	Added with plugins.	Provided by the surrounding framework.
Showing weaknesses	Easy to write a weak version, and to see exactly what is fixed in the hardened one.	Feasible, but validation and stricter defaults make a weak version less natural.	Feasible, but more setup before the first test.
Fit for this scope	Strong fit for a small REST API.	Feasible, with less familiar tooling.	Feasible, but less aligned with the time available.

The other components follow from the project constraints. React with TypeScript gives a realistic browser interface and a real surface for cross-site scripting tests. PostgreSQL provides a realistic relational database for SQL injection tests. Docker and Docker Compose make both versions reproducible, and allow isolated networks [5]. Ubuntu is a common server system that suits UFW and log inspection [7]. Nginx handles the reverse proxy and TLS in Version 2 [6]. Nmap, Wireshark, tcpdump and curl are fundamental tools that cover the network and application tests [8, 9].

4 Proposed solution

4.1 Application structure

Use React and TypeScript for the frontend, calling a REST API built with Express and TypeScript. Separate responsibilities in the backend into routes, controllers, services and repositories, as implemented in both versions: an authController, authService and userRepository for accounts, and a noteController, noteService and noteRepository for notes, written as TypeScript modules of functions rather than classes. An authenticate middleware (AuthMiddleware) protects all note routes, while register and login remain public. Login uses server-side sessions identified by a cookie (express-session). The database has two application tables, users and notes, related one to many, with the same columns in both versions. Version 2 adds constraints, valid timestamp defaults and a table for sessions stored in PostgreSQL, while Version 1 keeps sessions in memory. Only the security behaviour differs: for example, Version 1 stores passwords in plain text for demonstration and Version 2 stores an Argon2id hash. The Design document refines this structure and shows how the components interact, rather than repeating the technology comparison.

The API follows REST conventions in both versions, so that the only differences between them are the security behaviours being tested. Notes are a resource: /api/notes is the collection and /api/notes/:id is one note, with GET to read, POST to create, PUT to update and DELETE to remove, and JSON in both directions. Status codes carry the result: 201 for created, 400 for invalid input, 401 when the user is not authenticated, 404 when a note is missing, 409 for a duplicate username and 429 when the login limit is exceeded. Version 2 deliberately returns the same 404 for a note that exists but belongs to another user, so the API does not reveal which note IDs exist. The register, login and logout endpoints (POST /api/auth/...) are action-style routes because they do not map to a stored resource, and /api/health, /api/csrf-token and /api/test-auth are utility endpoints rather than resources. One deliberate deviation is that sessions are kept on the server and identified by a cookie, so the API is not strictly stateless. This is a trade-off, because session handling and cookie flags are among the planned tests. The API table in the Design document should be kept in step with these rules, and the /api/test-auth debug endpoint should be removed from Version 2 before deployment.

The backend deliberately does not use domain classes, meaning entity classes that carry their own data and behaviour as in domain-driven design. SecureNotes has only two entities, users and notes, and its business rules are limited to authentication and note ownership. Domain classes would add structure without adding behaviour, because the same logic would only move out of the services. Instead, the services and repositories are plain functions, database rows travel through the layers as plain objects, and the rules sit in the services and in the SQL conditions. This keeps the code short and makes it easy to see exactly where each weakness is introduced and fixed, for example the id and user_id condition that Version 2 adds to every read, update and delete in noteRepository. It also keeps both versions structurally identical, so a comparison shows only the security changes. Domain classes would become worthwhile only if the scope grew with features such as roles or note sharing, which are outside the baseline.

4.2 Environments and deployment

Run Version 1 in an Ubuntu virtual machine in the Fontys ICT Netlab with Docker, over plain HTTP and with a deliberately weak configuration. Netlab access is through the Fontys VPN and the VMware vCenter portal, using a Fontys account and the Netlab account supplied by the teacher [11]. Run Version 2 on the Ubuntu VPS with Docker Compose, Nginx with TLS, and UFW allowing only ports 22, 80 and 443. The

API ports (3000 in Version 1 and 3001 in Version 2) and PostgreSQL (5432) must not be public, and PostgreSQL stays on the internal Docker network in Version 2. Version 1 publishes PostgreSQL on port 5432 with a weak password, which is another reason it must stay isolated. The intentionally vulnerable version must never be left open to the public internet. If Netlab access or its rules prevent a planned test, a local virtual machine or an isolated Docker network is the fallback.

4.3 Security testing and hardening approach

Write the test cases before attacking, and run the same cases against both versions. Use Nmap for the port scan, Wireshark or tcpdump for traffic capture, curl and browser developer tools for authentication, authorization, injection and cross-site scripting tests, and the server logs to check what was recorded. For each finding, record how it was reproduced, why it is a security problem, how it was fixed and what the retest showed.

The fixes for Version 2 are implemented in the repository but must still be tested with the shared test cases, not assumed to work: Argon2id hashing, parameterized queries, server-side validation and safe output rendering, security headers with Helmet including a Content-Security-Policy, ownership checks on every note query, a CSRF token on state-changing requests, a single allowed CORS origin, a new session ID at login, HTTPS with Nginx, UFW rules, no public database port, rate limiting on login, security logging, secure cookies, and generic error messages for clients [2, 10]. Express does not include schema validation, so Version 2 uses explicit checks in the controllers for types, lengths, allowed fields and positive integer IDs, and these must be tested deliberately. Security logging of failed logins and denied access is still to do, because the code currently logs only server errors.

CSRF protection was not in the weakness list of Locksmith Lab Analysis, so it is an added finding and needs its own test case. Cross-site scripting needs a decision: React escapes note text by default and neither frontend currently inserts HTML, so a stored XSS test only works if Version 1 deliberately renders note content as HTML. Otherwise the result should be documented as not exploitable, with the reason.

4.4 Priority order

Implement all Must haves first: the working SecureNotes application, an isolated Version 1, the planned tests with evidence, the fixes in Version 2, the retest and the VPS deployment. Then add the Should haves that strengthen Version 2, starting with rate limiting (S01) and security logging (S02). Security headers (S03), cookie settings (S04) and SSH hardening (S05) follow if the core passes validation. Fail2ban and automated test scripts are Could haves and can be omitted. Note sharing, file uploads, multiple user roles and multi-factor authentication remain outside the baseline.

At the time of writing, rate limiting (S01), security headers (S03) and cookie settings (S04) exist in the Version 2 code and still need to be verified with the test cases, while security logging (S02), SSH hardening (S05) and the Nginx and VPS deployment remain to be done.

5 Quality and Optional Requirements

The table below lists quality and priority (Should/Could) requirements that support the proposed solution above, particularly the priority order (4.4) and the environments and deployment (4.2). The Should requirements S01 to S05 correspond to REQ-15 to REQ-19 in Locksmith Lab Analysis.

ID and priority	Requirement	Acceptance check
Q01 Must	Keep both versions reproducible.	Rebuild each version from the repository with Docker Compose on a clean machine or VM. The application starts and passes the basic functional checks.

ID and priority	Requirement	Acceptance check
Q02 Must	Keep secrets out of the repository.	Inspect the repository and history. No passwords, keys or tokens are committed. Configuration is read from environment settings.
Q03 Must	Run the same tests on both versions.	Compare the test record for Version 1 and Version 2. Every test case has a result for each version.
S01 Should	Rate limit login attempts.	Send repeated failed logins with curl. Version 2 limits the attempts and Version 1 does not.
S02 Should	Log security events.	Trigger failed logins and denied note access, then check the logs. Each event is recorded with a time and no password is stored in the log.
S03 Should	Set security headers.	Inspect the response headers in browser tools. Content-Security-Policy and HSTS are present on Version 2.
S04 Should	Use secure cookie settings.	Inspect the session cookie. HttpOnly, Secure and SameSite are set on Version 2.
S05 Should	Harden SSH on the VPS.	Confirm that password login is disabled and key authentication works.
C01 Could	Use Fail2ban on the VPS.	If included, repeated failed SSH logins lead to a temporary ban.
C02 Could	Automate the test cases with a script.	If included, the script repeats the curl-based tests and reports pass or fail for each version.

6 Design and implementation evidence

The Design portfolio evidence should include the architecture diagrams for Version 1 and Version 2, the database ERD, the backend UML or component diagram with AuthMiddleware, the API table, the authentication sequence diagram, a Version 1 versus Version 2 security comparison table, and a security test design. Each design should support an Analysis requirement. For example, the architecture diagrams should show the network boundary and the exposed ports, and the authentication sequence should show where password hashing, cookies and HTTPS apply in Version 2.

The Realization evidence should contain the source code, setup and run instructions, and an explanation of the tools used. Build and test in small increments, starting with a running container stack, then registration and login, then notes, and then each weakness or fix. Record meaningful differences between the final implementation and the original design.

The source code is kept in a Git repository on GitHub (https://github.com/N1ki2K/Locksmith-Lab) so the evidence can be reviewed and rebuilt. It has a docs folder with the project documents, diagrams, API table and realisation log, and separate unsecured and secured folders, each with its own API, frontend, database script and Docker Compose file. The setup needs Git, Node.js, npm and Docker Compose:

- Clone the repository, start the database of a version with docker compose up -d database from its folder (unsecured or secured), then start the API and the frontend with npm run dev. Containerising the API and frontend, so that a single docker compose up --build starts everything on the VPS, is still planned.
- Each version has its own Compose file, container, volume and ports, so the two can be started independently or side by side locally: Version 1 uses API port 3000, database port 5432 and frontend port 5173, and Version 2 uses 3001, 5433 and 5174.

- Version 2 keeps its configuration in secured/.env.example with placeholders only, and real secrets stay out of Git. Version 1 intentionally uses hard-coded weak credentials.

7 Validation and completion

Use the requirement identifiers in Locksmith Lab Analysis to create a test record with test steps, expected behavior, actual behavior, pass or fail status, and evidence such as command output, screenshots and packet captures. Record the environment (Netlab VM or VPS, operating system and tested build). Cover at least the following tests on both versions: port scan, HTTP capture, SQL injection, broken access control, cross-site scripting, repeated login attempts and log review. Version 2 already has automated integration tests in secured/api/test for registration and hashing, SQL injection strings, CSRF, session rotation, logout, note ownership and invalid input. These support the shared test record but do not replace it, because every test case needs a result for both versions.

Ask a peer or teacher to review the test cases before the tests are run, and to check that the environments are isolated as agreed. A teacher review can check the technical approach and portfolio coverage. Completion requires every Must have to pass, every documented finding to have a fix and a retest result, a Version 2 whose exposed ports and HTTPS are verified from outside the VPS, and reproducible source code with supporting evidence.

8 Use of AI during the project

If I use generative AI, I will use it to explain concepts, review small sections of code and help investigate errors. I will check technical suggestions against documentation and test each change before relying on it. I need to be able to explain the code, the vulnerabilities and the fixes in my portfolio. Attack commands and payloads suggested by AI are only run inside the isolated test environment, and never against systems I do not own or that are not provided for this project.

For any AI-assisted implementation, I will retain relevant prompts or inputs, explain what I changed, and record how I checked the result. These are planned working practices and should be supported by actual evidence as development proceeds.

9 Sources

Technical references (access dates to be added when the links are checked):

#	Reference
[1]	Express. Documentation. https://expressjs.com/
[2]	Express. Production best practices: Security. https://expressjs.com/en/advanced/best-practice-security.html
[3]	Fastify. Validation and Serialization. https://fastify.dev/docs/latest/Reference/Validation-and-Serialization/
[4]	Spring. Spring Boot. https://spring.io/projects/spring-boot
[5]	Docker. Docker Compose documentation. https://docs.docker.com/compose/
[6]	Nginx. Configuring HTTPS servers. https://nginx.org/en/docs/http/configuring_https_servers.html
[7]	Ubuntu. UFW. https://help.ubuntu.com/community/UFW
[8]	Nmap Project. Nmap Reference Guide. https://nmap.org/book/man.html
[9]	Wireshark Foundation. Wireshark User’s Guide. https://www.wireshark.org/docs/
[10]	OWASP Foundation. OWASP Top 10. https://owasp.org/Top10/

#	Reference
[11]	Fontys ICT. EduCloud documentation for the ICT Netlab (link to be added).